

EE789:Algorithmic Design of Digital Systems

Assignment 1 Report

Name of student : Prathmesh Baral

Roll no : 23B1260

March 21, 2026

1 Problem Statement

The objective of this project is to design and analyze a 2×2 packet switch architecture. The switch consists of two input ports and two output ports, and is responsible for routing incoming data packets to the appropriate output port based on the destination address specified in the packet header.

Data enters the switch in the form of packets, where each packet is composed of multiple words. Each word is 4 bytes in size, and packets may contain a variable number of words.

Each packet is preceded by a header word that contains routing and control information. The header format is defined as follows:

- Bits [31:24]: Destination address (e.g., 1 for Output Port 1, 2 for Output Port 2)
- Bits [23:8]: Packet length (in number of words, including the header)
- Bits [7:0]: Sequence ID

The switch must correctly interpret the header information and ensure that packets are forwarded to the intended output port while maintaining data integrity.

As part of this project, three different switching architectures are to be designed and evaluated:

1. Shared Queue Switch
2. Input-Queued Switch
3. Output-Queued Switch

Each architecture will be analyzed in terms of its functionality, buffering strategy, and performance characteristics.

2 Shared Queued Switch Architecture

The shared queued switch architecture is designed such that packets arriving at the input ports are first stored in a common queue before being forwarded to the appropriate output ports. The design is divided into the following major components:

- Input Arbiter (Input Taker)
- FIFO Queue
- Queue Flag Register
- Output Demultiplexer

2.1 Input Arbiter (Input Taker)

The input arbiter is responsible for receiving packets from the input ports and loading them into the queue. It is implemented using a Finite State Machine (FSM) with four states:

1. **WAIT_Header**
2. **Send_packet_from_1**
3. **Send_packet_from_2**
4. **WAIT_Buffer**

State Descriptions:

- **WAIT_Header:** In this state, the arbiter waits for a request signal from either input port.
 - If `in1_req` is asserted:
 - * Next state $\rightarrow$ `Send_packet_from_1`
 - * Packet count $\leftarrow$ `in1_data[23:8]`
 - * Queue add signal (`q_add`) is asserted
 - If `in2_req` is asserted:
 - * Next state $\rightarrow$ `Send_packet_from_2`
 - * Packet count $\leftarrow$ `in2_data[23:8]`
 - * Queue add signal (`q_add`) is asserted
- **Send_packet_from_1:** The FSM locks onto Input Port 1 and continuously transfers packet data into the queue.
 - Packet count is decremented for each word:
$$\text{pkt_count} \leftarrow \text{pkt_count} - 1$$
 - Queue input:
$$\text{queue_in} \leftarrow \text{dest} \parallel \text{in1_data}$$
 - If `pkt_count = 1`, transition to `WAIT_Buffer`
 - Else, remain in the same state
- **Send_packet_from_2:** Similar to the previous state, but operates on Input Port 2.

- Packet count is decremented:

$$\text{pkt_count} \leftarrow \text{pkt_count} - 1$$

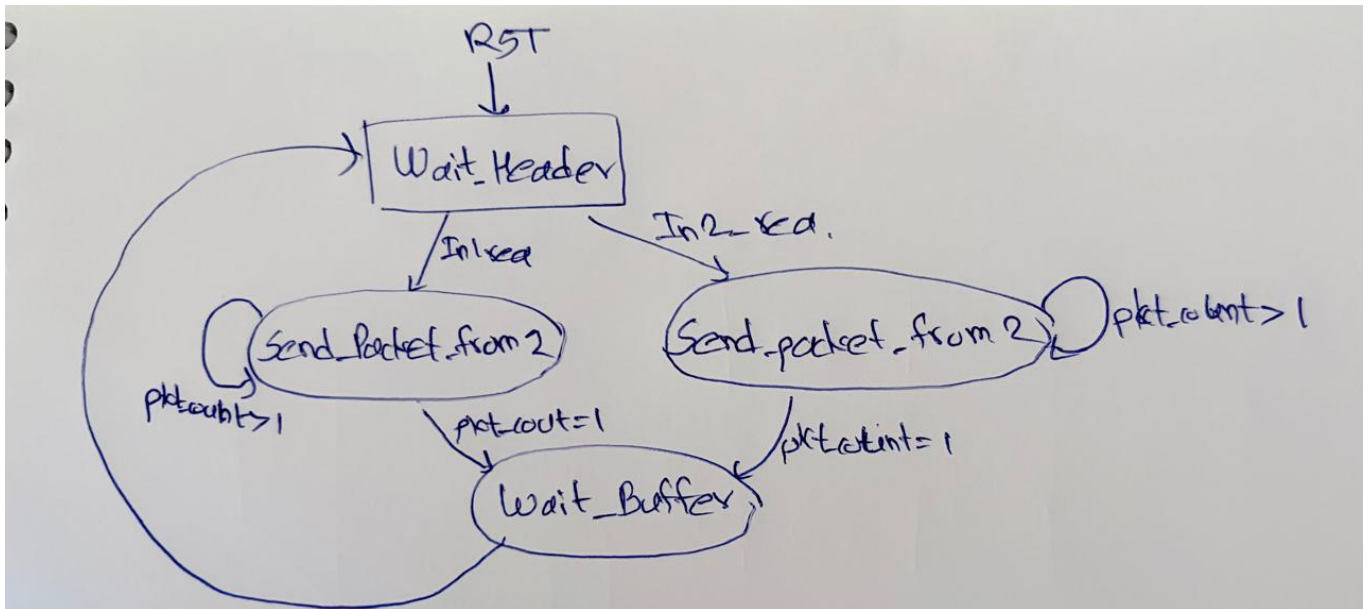
- Queue input:

$$\text{queue_in} \leftarrow \text{dest} \parallel \text{in2_data}$$

- If $\text{pkt_count} = 1$, transition to `WAIT_Buffer`
- Else, remain in the same state

- **WAIT_Buffer:** This state ensures proper buffering and resets control signals before accepting a new packet.
 - Next state $\rightarrow$ `WAIT_Header`

Working Principle: Once a request is detected from any input port, the FSM locks onto that port and transfers the entire packet into the queue based on the packet length field. During this process, other input requests are ignored. When the packet transfer completes, the FSM moves to the `WAIT_Buffer` state, after which it returns to `WAIT_Header` to process the next packet.



2.2 FIFO Queue

A FIFO-based storage structure is used to buffer incoming packets. The queue has the following characteristics:

- Word size: 33 bits (32-bit data + 1-bit destination field)
- Depth: 1024 words

Each word stored in the queue contains the original data along with the destination information, which is later used for routing.

2.3 Queue Flag Register

A flag register is implemented to keep track of the number of packets stored in the queue.

- When the number of stored packets is greater than zero, a `valid` signal is asserted, indicating that the queue can be read.

- The register is updated as follows:
 - Incremented when a packet is added (triggered by the input arbiter)
 - Decrementd when a packet is transmitted (triggered by the output demultiplexer)

2.4 Output Demultiplexer

The output demultiplexer is responsible for forwarding packets from the queue to the appropriate output port based on the destination field. It is implemented using a Finite State Machine (FSM) along with a packet counter.

The FSM consists of the following four states:

1. **WAIT_OUT**
2. **Send_packet_to1**
3. **Send_packet_to2**
4. **WAIT_BUFFER**

State Descriptions:

- **WAIT_OUT:** The FSM waits until valid data is available in the queue.
 - If `q_valid` is 1:
 - * If `out1_req` is asserted and `destination = 1`:
 - Next state $\rightarrow$ `Send_packet_to1`
 - Packet count $\leftarrow$ `q_data[23:8]`
 - Queue subtraction trigger (`q_sub`) is asserted
 - * If `out2_req` is asserted and `destination = 2`:
 - Next state $\rightarrow$ `Send_packet_to2`
 - Packet count $\leftarrow$ `q_data[23:8]`
 - Queue subtraction trigger (`q_sub`) is asserted
- **Send_packet_to1:** In this state, packet data is transmitted to Output Port 1.
 - Output assignment:

$$\text{out1} \leftarrow \text{q_data}$$
 - Packet count is decremented:

$$\text{pkt_count} \leftarrow \text{pkt_count} - 1$$
 - If `pkt_count = 1`, transition to `WAIT_BUFFER`
- **Send_packet_to2:** In this state, packet data is transmitted to Output Port 2.
 - Output assignment:

$$\text{out2} \leftarrow \text{q_data}$$
 - Packet count is decremented:

$$\text{pkt_count} \leftarrow \text{pkt_count} - 1$$
 - If `pkt_count = 1`, transition to `WAIT_BUFFER`

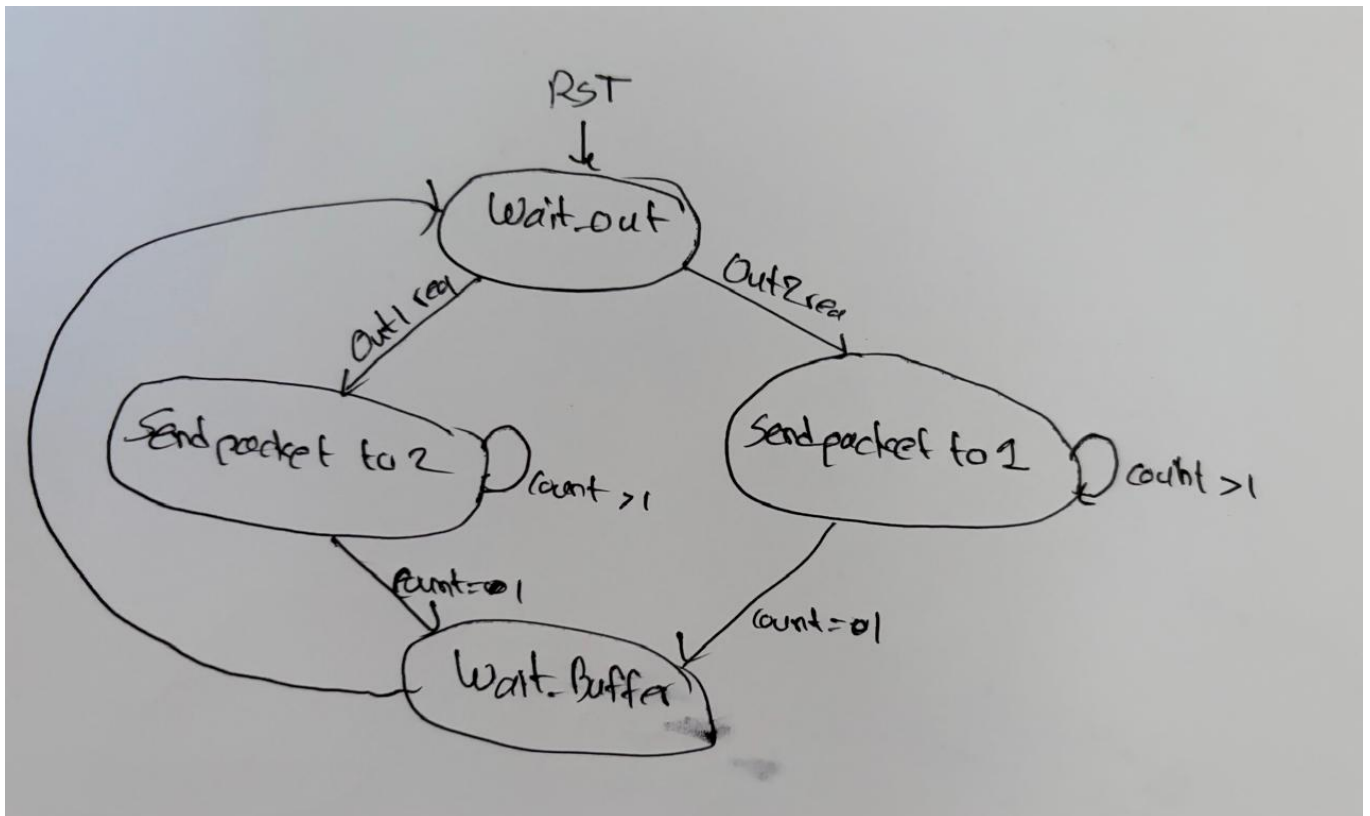
- **WAIT_BUFFER:** This state provides a buffer cycle to reset control signals before processing the next packet.

– Next state → WAIT_OUT

Working Principle: The output demultiplexer continuously monitors the queue. When valid data is available (`q_valid`), it checks whether any output port has requested data. If a match occurs between the requested port and the destination field at the head of the queue, the FSM starts transmitting the packet.

The packet header is read only when the queue is valid. Once transmission begins, the FSM locks onto the selected output port and continues sending the packet until completion. After the entire packet is transmitted, the FSM moves to the WAIT_BUFFER state before returning to WAIT_OUT.

Additionally, a trigger signal is generated whenever a packet is removed from the queue, which updates the queue flag register.



2.5 Top-Level Module

The top-level module integrates all the submodules of the input queued switch, including:

- Input Arbiter
- FIFO Queue
- Queue Flag Register
- Output Demultiplexer

These modules are interconnected to ensure seamless data flow from input ports to output ports. The input arbiter writes incoming packets into the queue, while the output demultiplexer reads packets from the queue and forwards them to the appropriate output port.

The queue flag register acts as an intermediary control unit, providing status signals that coordinate read and write operations.

The overall system ensures that packets are handled sequentially, maintaining data integrity and correct routing based on the destination field.

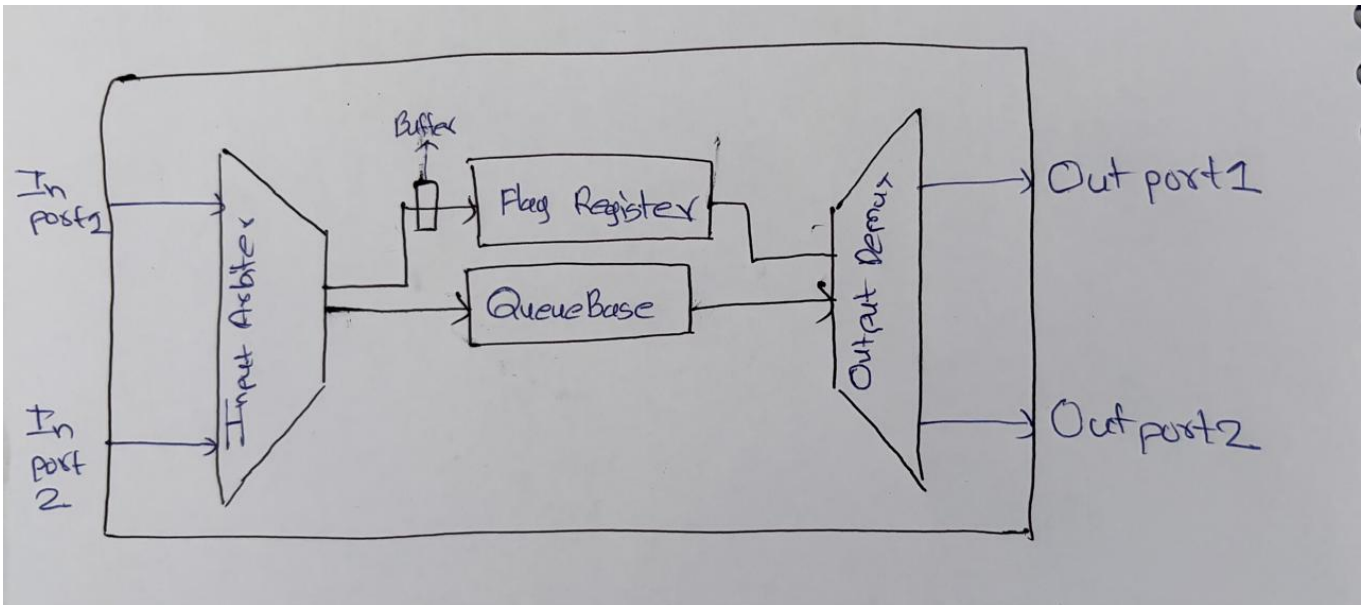


Figure 1: Block diagram of the input queued switch showing interconnection of all modules

3 Input-Queued Switch Architecture

In this architecture, each input port is associated with an independent queue. Incoming packets are first stored in their respective input queues before being forwarded to the appropriate output port.

The design consists of the following modules:

- Input Taker
- FIFO Queues (one per input)
- Flag Registers
- Output Demultiplexer
- Output Multiplexer

3.1 Input Taker

The input taker is responsible for receiving packets from each input port and storing them in the corresponding input queue. It is implemented using a Finite State Machine (FSM) with three states:

1. **WAIT_REQ**
2. **LOAD_PACKET**
3. **WAIT_BUFFER**

State Descriptions:

- **WAIT_REQ:** The FSM waits for a request signal from the input port.
 - If `in_req` is asserted:
 - * `Packet count` $\leftarrow$ `in_data[23:8]`
 - * `Destination` $\leftarrow$ extracted from header
 - * Add trigger is sent to the flag register

* Next state $\rightarrow$ LOAD_PACKET

- **LOAD_PACKET:** In this state, the packet data is written into the corresponding input queue.

– Queue input:

$$q_in \leftarrow dest \parallel in_data$$

– Packet count is decremented:

$$pkt_count \leftarrow pkt_count - 1$$

– If $pkt_count = 1$, transition to WAIT_BUFFER

- **WAIT_BUFFER:** This state provides a buffer cycle to stabilize control signals before accepting a new packet.

– Next state $\rightarrow$ WAIT_REQ

Working Principle: As soon as a packet arrives at an input port, the FSM captures the header, extracts the packet length and destination, and begins loading the packet into the corresponding input queue. Each packet is stored along with its destination field. A trigger signal is generated to update the flag register whenever a new packet is added. After the packet is fully stored, the FSM returns to the idle state through a buffer cycle.

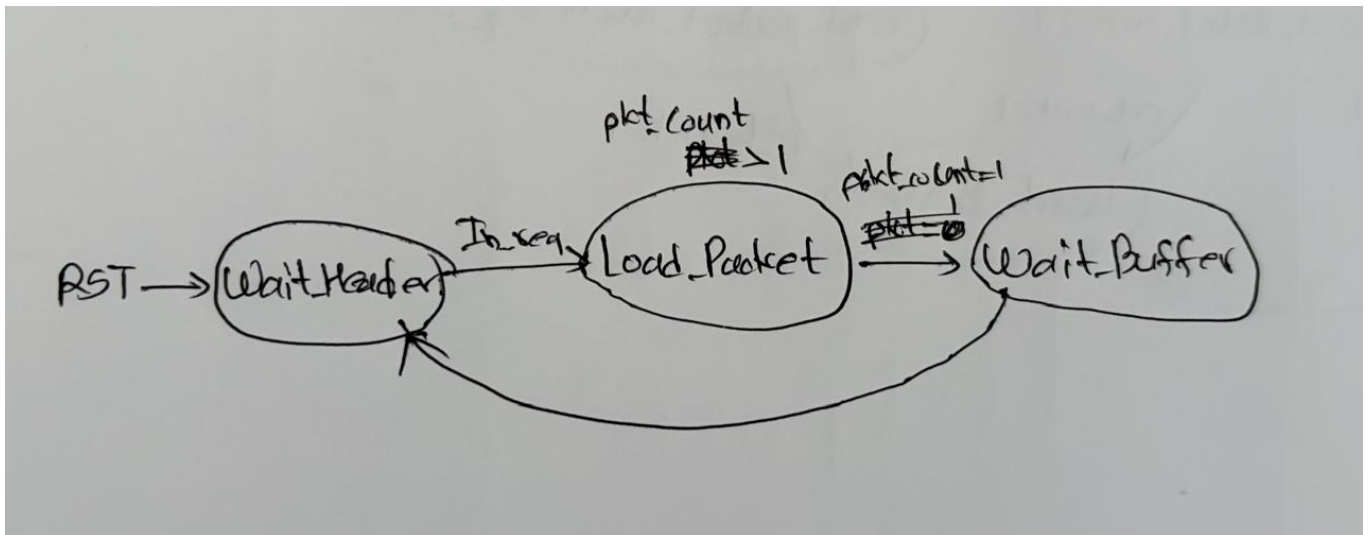


Figure 2: FSM of Input Taker

3.2 FIFO Queues

Each input port has an independent FIFO queue for buffering packets. The characteristics are as follows:

- Word size: 33 bits (32-bit data + 1-bit destination)
- Separate queue per input port

This separation avoids contention at the input stage and allows parallel packet storage.

3.3 Flag Registers

Flag registers are used to track the number of packets stored in each input queue.

- A `valid` signal is asserted when the queue contains one or more packets
- Incremented when a packet is added (input taker trigger)
- Decrementd when a packet is removed (output side trigger)

3.4 Output Demultiplexer

The output demultiplexer is connected to the output of each input queue. Its function is to route packet data toward the appropriate output port based on the destination field.

The module is implemented using an FSM with the following states:

1. **WAIT_REQ**
2. **SEND_PACKET_TO1**
3. **SEND_PACKET_TO2**
4. **WAIT_BUFFER**

State Descriptions:

- **WAIT_REQ:** The FSM monitors the queue output.
 - Reads the data at the head of the queue
 - Asserts a `data_ready` signal to the output mux
 - Upon receiving an output request:
 - * If request is for Output Port 1 → transition to **SEND_PACKET_TO1**
 - * If request is for Output Port 2 → transition to **SEND_PACKET_TO2**
- **SEND_PACKET_TO1:** Packet data from the queue is forwarded toward Output Port 1 via the output mux.
- **SEND_PACKET_TO2:** Packet data from the queue is forwarded toward Output Port 2 via the output mux.
- **WAIT_BUFFER:** A buffer state used to reset control signals before processing the next packet.

Working Principle: Whenever the queue contains valid data, the demultiplexer reads the packet header and determines the destination. It then asserts a ready signal toward the output mux corresponding to that destination.

The actual transmission begins only after the corresponding output port issues a request. Once enabled, the demultiplexer forwards the entire packet. At the completion of packet transmission (last word), a `done` trigger is generated, and the FSM transitions to the **WAIT_BUFFER** state before returning to **WAIT_REQ**.

This mechanism ensures controlled and conflict-free data flow from queues to output ports.

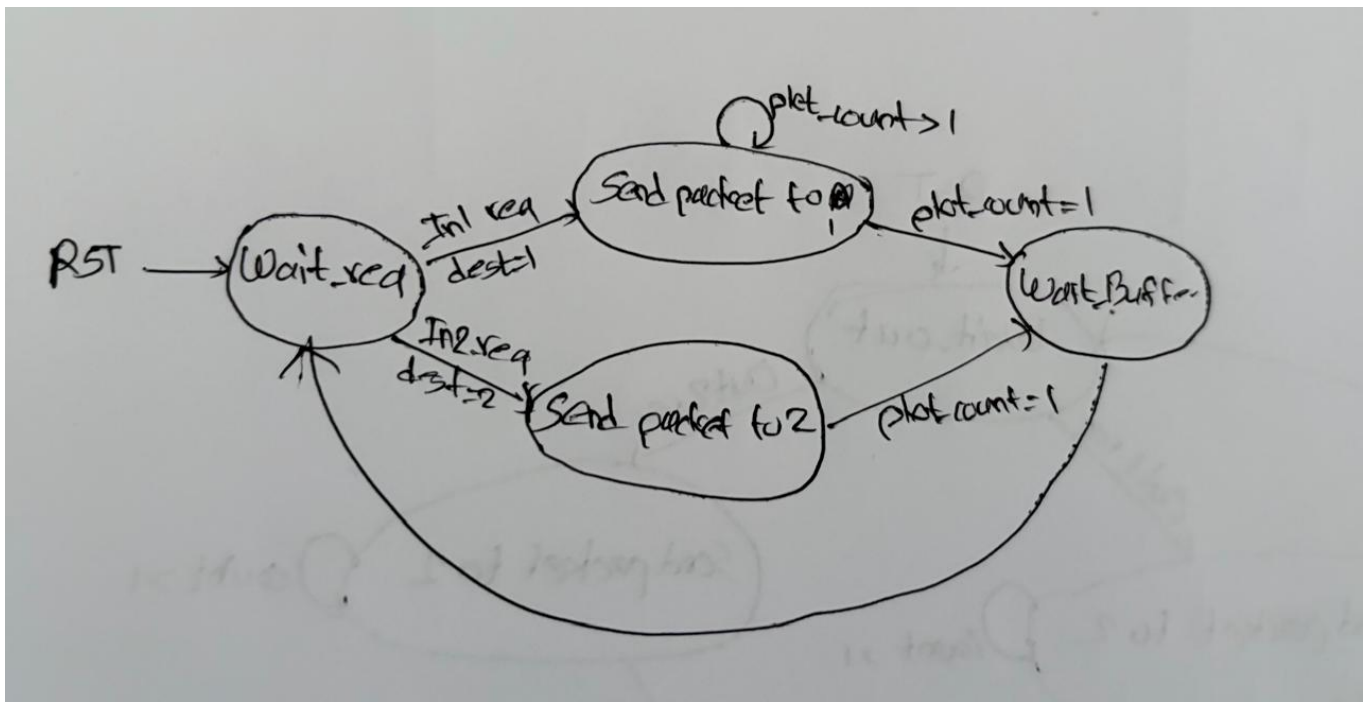


Figure 3: FSM of Output Mux

3.5 Output Multiplexer

The output multiplexer is connected at each output port and is responsible for selecting between multiple input queues that may request access to the same output port.

Each output port has one dedicated multiplexer.

The FSM for the multiplexer consists of the following states:

1. **WAIT_REQ**
2. **SEND_DATA_FROM1**
3. **SEND_DATA_FROM2**
4. **WAIT_BUFFER**

State Descriptions:

- **WAIT_REQ:** The multiplexer waits for a request from the output port.
 - If `out_req` is asserted:
 - * If Queue 1 has valid data (`q1_ready`), transition to **SEND_DATA_FROM1**
 - * Else if Queue 2 has valid data (`q2_ready`), transition to **SEND_DATA_FROM2**
- **SEND_DATA_FROM1:** The multiplexer establishes communication between Queue 1 (via its de-mux) and the output port.
 - Data is forwarded continuously
 - Upon receiving a `done` signal, transition to **WAIT_BUFFER**
- **SEND_DATA_FROM2:** Similar to the previous state, but selects Queue 2 as the data source.
 - Upon receiving a `done` signal, transition to **WAIT_BUFFER**
- **WAIT_BUFFER:** Provides a buffer cycle before returning to **WAIT_REQ**

Working Principle: When an output request is received, the multiplexer checks which input queue has valid data destined for that output. It then locks the communication with that queue and forwards the packet.

Once the full packet is transmitted, the demultiplexer signals completion using a done trigger. The multiplexer then transitions through a buffer state before accepting the next request.

This mechanism resolves contention between multiple input queues competing for the same output port.

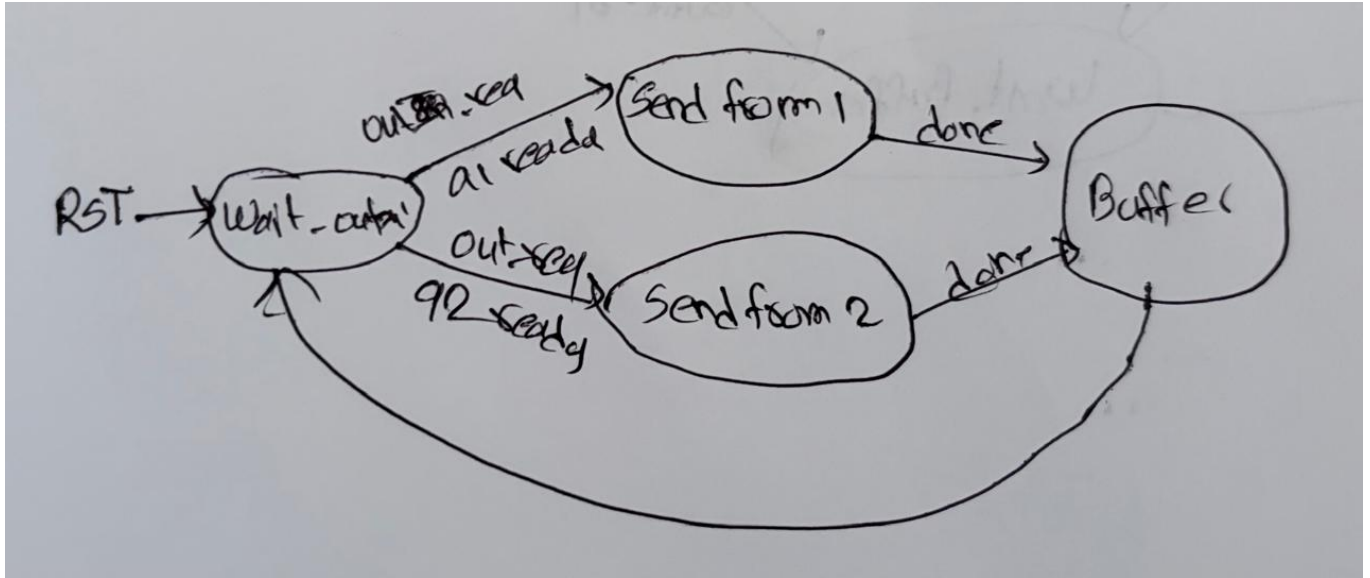


Figure 4: FSM of Output Mux

3.6 Top-Level Module

The top-level module integrates all components of the input-queued switch with per-input buffering. The design includes:

- Two Input Takers (one per input port)
- Two FIFO Queues
- Two Flag Registers
- Two Output Demultiplexers
- Two Output Multiplexers (one per output port)

Interconnection Description:

- Input ports are connected to their respective input takers
- Each input taker feeds its corresponding queue and flag register
- The output of each queue is connected to its output demultiplexer
- Outputs of the demultiplexers are connected to the input of output multiplexers
- Each output multiplexer is connected to a corresponding output port

This architecture ensures that packets are independently buffered at each input and are routed efficiently to the correct output while handling contention through multiplexing.

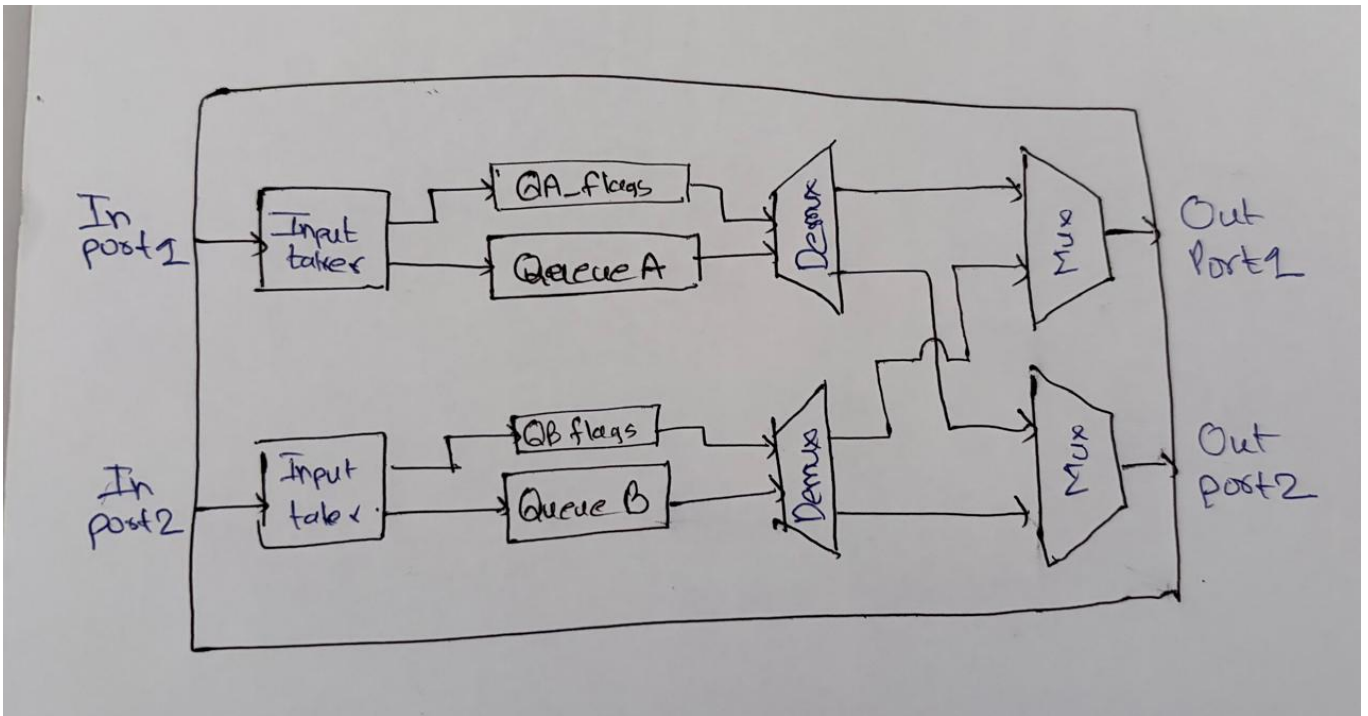


Figure 5: Top-level architecture of input-queued switch with per-input queues

4 Output-Queued Switch Architecture

In the output-queued switch architecture, packets are directly routed to queues associated with output ports at the time of arrival. Each output port maintains its own queues from every input port, eliminating the need to store destination information within the queue.

The design consists of the following modules:

- Input Demultiplexer
- FIFO Queues (per output)
- Flag Registers
- Output Multiplexer

4.1 Input Demultiplexer

The input demultiplexer is connected to each input port and routes incoming packets to the appropriate output queue based on the destination field. It is implemented using a Finite State Machine (FSM) with the following states:

1. **WAIT_REQ**
2. **SEND_PACKET_TO1**
3. **SEND_PACKET_TO2**
4. **WAIT_BUFFER**

State Descriptions:

- **WAIT_REQ:** The FSM waits for an input request.

- When `in_req` is asserted:
 - * `Packet count` $\leftarrow$ `in_data[23:8]`
 - * Destination is decoded from header
 - * If destination = 1:
 - Next state $\rightarrow$ `SEND_PACKET_TO1`
 - Add trigger is given to Queue 1 flag register
 - * If destination = 2:
 - Next state $\rightarrow$ `SEND_PACKET_TO2`
 - Add trigger is given to Queue 2 flag register
- **SEND_PACKET_TO1:** Packet data is written into the queue corresponding to Output Port 1.
 - `q1_data` $\leftarrow$ `in_data`
 - Packet count is decremented:

$$\text{pkt_count} \leftarrow \text{pkt_count} - 1$$
 - If `pkt_count` = 1, transition to `WAIT_BUFFER`
- **SEND_PACKET_TO2:** Packet data is written into the queue corresponding to Output Port 2.
 - `q2_data` $\leftarrow$ `in_data`
 - Packet count is decremented:

$$\text{pkt_count} \leftarrow \text{pkt_count} - 1$$
 - If `pkt_count` = 1, transition to `WAIT_BUFFER`
- **WAIT_BUFFER:** A buffer state used to reset control signals before accepting the next packet.

Working Principle: As soon as a packet arrives at the input port, the demultiplexer extracts the packet length and destination from the header. Based on the destination, the FSM transitions to the appropriate state and begins loading the packet into the corresponding output queue. A trigger signal is generated to update the flag register. After the packet is fully stored, the FSM moves through a buffer state before returning to the idle state.

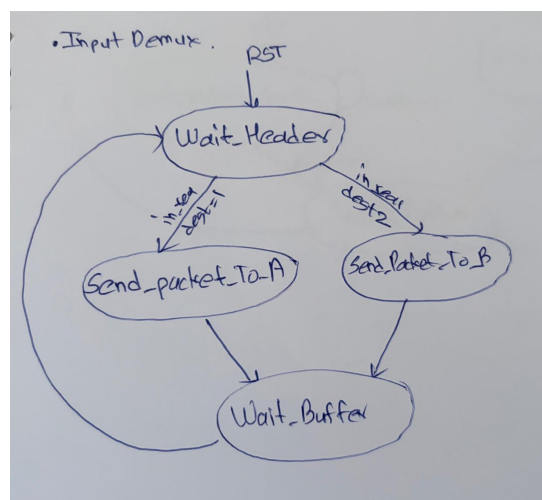


Figure 6: FSM transitions of input demux

4.2 FIFO Queues

Each output port has dedicated queues to store incoming packets. Since packets are already routed to their correct destination, there is no need to store the destination field within the queue.

- Word size: 32 bits
- Queue depth: 256 words

4.3 Flag Registers

Flag registers are used to track the number of packets stored in each queue.

- A `valid` signal indicates the presence of packets in the queue
- Incremented when a packet is added (input demux trigger)
- Decrementd when a packet is transmitted (output mux trigger)

4.4 Output Multiplexer

The output multiplexer connects multiple queues to a single output port and resolves contention between them.

The FSM consists of the following states:

1. **WAIT_REQ**
2. **SERVE_Q1**
3. **SERVE_Q2**
4. **WAIT_BUFFER**

State Descriptions:

- **WAIT_REQ:** The FSM waits for a request from the output port.
 - If `out_req` is asserted:
 - * If Queue 1 is valid:
 - Next state $\rightarrow$ **SERVE_Q1**
 - Packet count $\leftarrow$ `q1_data[23:8]`
 - Subtract trigger is given to Queue 1 flag register
 - * Else if Queue 2 is valid:
 - Next state $\rightarrow$ **SERVE_Q2**
 - Packet count $\leftarrow$ `q2_data[23:8]`
 - Subtract trigger is given to Queue 2 flag register
- **SERVE_Q1:** Data is forwarded from Queue 1 to the output port.
 - `out_data` $\leftarrow$ `q1_data`
 - Packet count is decremented
 - If `pkt_count` = 1, transition to **WAIT_BUFFER**
- **SERVE_Q2:** Data is forwarded from Queue 2 to the output port.
 - `out_data` $\leftarrow$ `q2_data`

- Packet count is decremented
- If `pkt_count = 1`, transition to `WAIT_BUFFER`
- **WAIT_BUFFER:** A buffer state before returning to `WAIT_REQ`

Working Principle: When an output request is received, the multiplexer checks which queue has valid data and selects it. The communication between the selected queue and the output port is maintained until the entire packet is transmitted. After completion, the FSM transitions through a buffer state before accepting the next request.

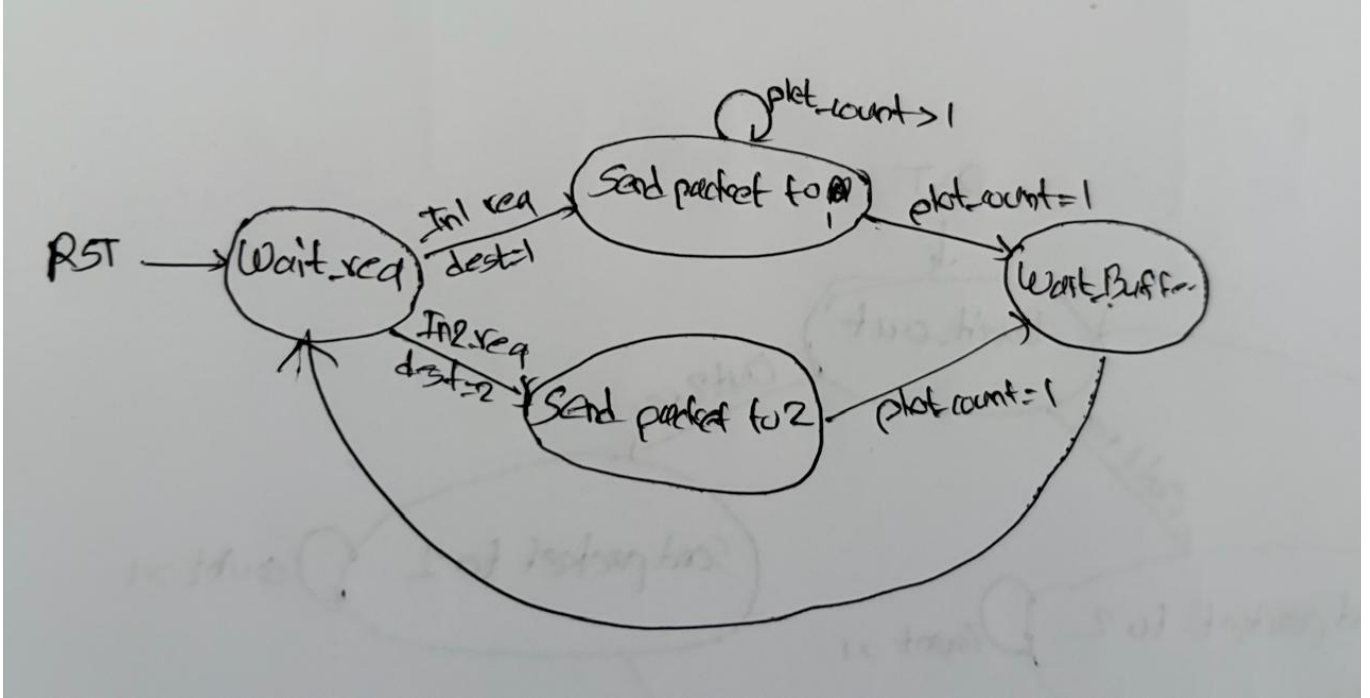


Figure 7: FSM transitions of output mux

4.5 Top-Level Module

The top-level implementation consists of:

- Two Input Demultiplexers (one per input port)
- Four FIFO Queues (two per output port)
- Four Flag Registers
- Two Output Multiplexers (one per output port)

Interconnection Description:

- Input Port 1 is connected to Demultiplexer 1, which routes data to Queue 1 and Queue 3
- Input Port 2 is connected to Demultiplexer 2, which routes data to Queue 2 and Queue 4
- Output Multiplexer 1 connects Queue 1 and Queue 2 to Output Port 1
- Output Multiplexer 2 connects Queue 3 and Queue 4 to Output Port 2

This structure ensures that packets are directly stored in output-specific queues, minimizing routing complexity at the output stage and improving throughput.

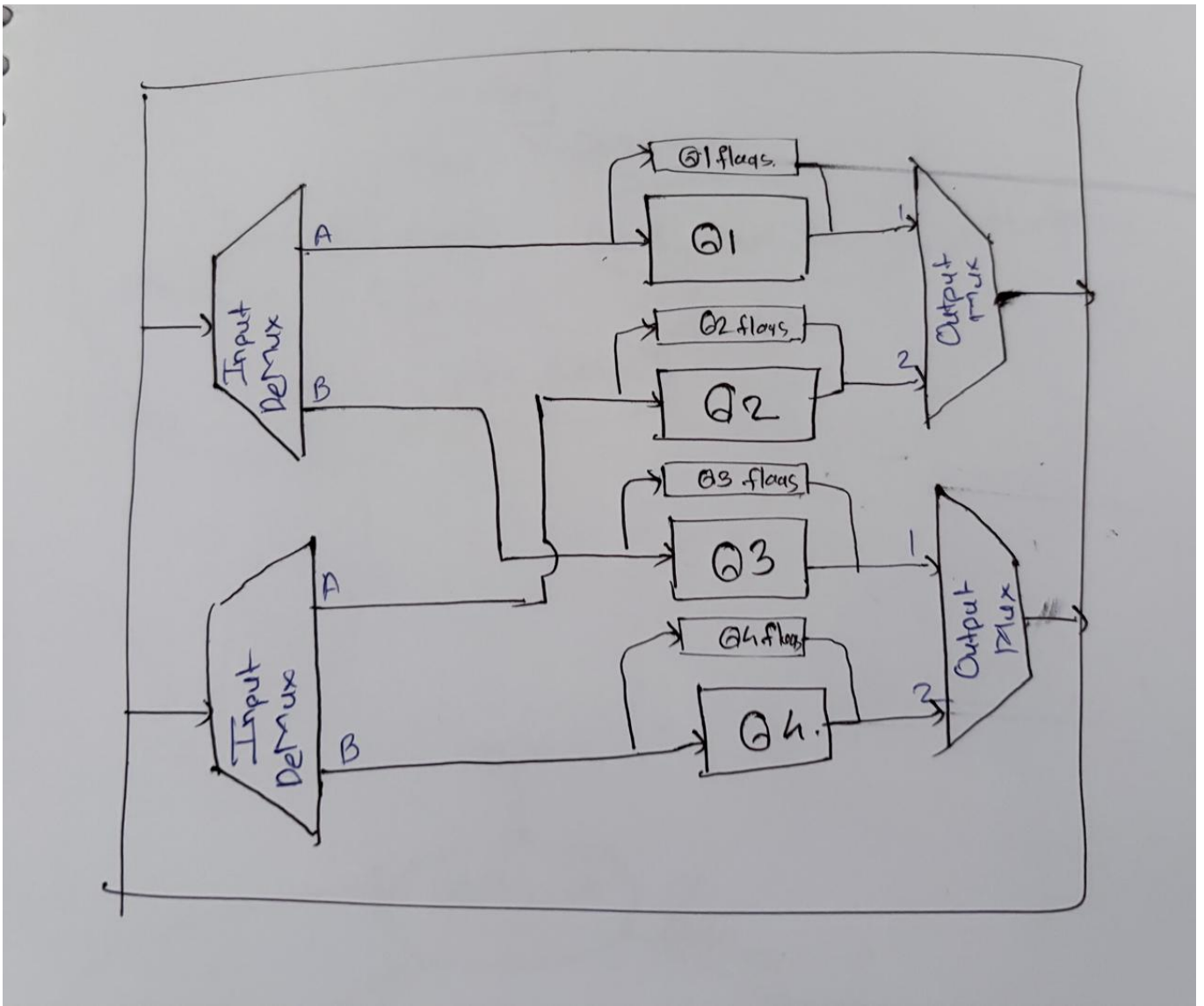


Figure 8: Top-level architecture of output-queued switch

5 Proof of Concept Video

A demonstration video showcasing the working of all three switch architectures has been recorded and uploaded.

The video includes:

- Testing of all implementations
- Each Implementation is tested on 1to1, 1to2, 1toBoth, BothtoBoth paths

The video can be accessed using the following link:

[Click here to view the demo video](#)

6 Testing of Shared queue switch

```
root@cf02f40d9b70:/home/EE789/switch_2x2_shared_queue# ./bin/testbench_vhdl st
dout 1to1 2>&1 | tee log1.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 1 from input port 1
Rx[2] at output port 1 from input port 1
Rx[3] at output port 1 from input port 1
Rx[4] at output port 1 from input port 1
Rx[5] at output port 1 from input port 1
Rx[6] at output port 1 from input port 1
Rx[7] at output port 1 from input port 1
Rx[8] at output port 1 from input port 1
```

```
Rx[218] at output port 1 from input port 1
Rx[219] at output port 1 from input port 1
Rx[220] at output port 1 from input port 1
Rx[221] at output port 1 from input port 1
Rx[222] at output port 1 from input port 1
Rx[223] at output port 1 from input port 1
Rx[224] at output port 1 from input port 1
Rx[225] at output port 1 from input port 1
Rx[226] at output port 1 from input port 1
Rx[227] at output port 1 from input port 1
Rx[228] at output port 1 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_shared_queue# |
```

Figure 9: Testing of 1to1 path

```
root@cf02f40d9b70:/home/EE789/switch_2x2_shared_queue# ./bin/testbench_vhdl st
dout 1toBoth 2>&1 | tee log2.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 2 from input port 1
Rx[1] at output port 1 from input port 1
Rx[2] at output port 2 from input port 1
Rx[3] at output port 2 from input port 1
Rx[4] at output port 2 from input port 1
Rx[5] at output port 2 from input port 1
Rx[2] at output port 1 from input port 1
Rx[3] at output port 1 from input port 1
```

```
Rx[245] at output port 1 from input port 1
Rx[246] at output port 1 from input port 1
Rx[247] at output port 2 from input port 1
Rx[215] at output port 2 from input port 1
Rx[216] at output port 2 from input port 1
Rx[248] at output port 1 from input port 1
Rx[217] at output port 2 from input port 1
Rx[218] at output port 2 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_shared_queue# |
```

Figure 10: Testing of 1toBoth path

```
root@cf02f40d9b70:/home/EE789/switch_2x2_shared_queue# ./bin/testbench_vhdl st
dout BothtoBoth 2>&1 | tee log4.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 2 from input port 1
Rx[2] at output port 2 from input port 1
Rx[3] at output port 2 from input port 1
Rx[4] at output port 2 from input port 1
Rx[5] at output port 2 from input port 1
Rx[1] at output port 1 from input port 1
Rx[2] at output port 1 from input port 1
Rx[6] at output port 2 from input port 1
Rx[7] at output port 2 from input port 1
Rx[3] at output port 1 from input port 1
Rx[8] at output port 2 from input port 1
```

```
Rx[273] at output port 1 from input port 1
Rx[274] at output port 1 from input port 1
Rx[275] at output port 1 from input port 1
Rx[276] at output port 1 from input port 2
Rx[277] at output port 1 from input port 1
Rx[278] at output port 1 from input port 1
Rx[279] at output port 1 from input port 2
Rx[280] at output port 1 from input port 1
Rx[281] at output port 1 from input port 1
Rx[250] at output port 2 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_shared_queue# |
```

Figure 11: Testing of BothtoBoth path

7 Testing of Input queued switch

```
root@cf02f40d9b70:/home/EE789/switch_2x2_input_queued# ./bin/testbench_vhdl st
dout 1to1 2>&1 | tee log1.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 1 from input port 1
Rx[2] at output port 1 from input port 1
Rx[3] at output port 1 from input port 1
Rx[4] at output port 1 from input port 1
Rx[5] at output port 1 from input port 1
Rx[6] at output port 1 from input port 1
Rx[7] at output port 1 from input port 1
```

```
Rx[262] at output port 1 from input port 1
Rx[263] at output port 1 from input port 1
Rx[264] at output port 1 from input port 1
Rx[265] at output port 1 from input port 1
Rx[266] at output port 1 from input port 1
Rx[267] at output port 1 from input port 1
Rx[268] at output port 1 from input port 1
Rx[269] at output port 1 from input port 1
Rx[270] at output port 1 from input port 1
Rx[271] at output port 1 from input port 1
Rx[272] at output port 1 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_input_queued# |
```

Figure 12: Testing of 1to1 path

```
root@cf02f40d9b70:/home/EE789/switch_2x2_input_queued# ./bin/testbench_vhdl st
dout 1toBoth 2>&1 | tee log3.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 2 from input port 1
Rx[1] at output port 1 from input port 1
Rx[2] at output port 2 from input port 1
Rx[3] at output port 2 from input port 1
Rx[4] at output port 2 from input port 1
Rx[5] at output port 2 from input port 1
Rx[2] at output port 1 from input port 1
Rx[3] at output port 1 from input port 1
```

```
Rx[185] at output port 2 from input port 1
Rx[209] at output port 1 from input port 1
Rx[186] at output port 2 from input port 1
Rx[187] at output port 2 from input port 1
Rx[188] at output port 2 from input port 1
Rx[210] at output port 1 from input port 1
Rx[211] at output port 1 from input port 1
Rx[212] at output port 1 from input port 1
Rx[189] at output port 2 from input port 1
Rx[213] at output port 1 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_input_queued# |
```

Figure 13: Testing of 1toBoth path

```
root@cf02f40d9b70:/home/EE789/switch_2x2_input_queued# ./bin/testbench_vhdl st
dout BothtoBoth 2>&1 | tee log4.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 2 from input port 1
Rx[1] at output port 1 from input port 2
Rx[2] at output port 2 from input port 1
Rx[3] at output port 2 from input port 1
Rx[2] at output port 1 from input port 1
Rx[4] at output port 2 from input port 2
Rx[5] at output port 2 from input port 1
Rx[6] at output port 2 from input port 1
```

```
Rx[316] at output port 1 from input port 2
Rx[317] at output port 1 from input port 2
Rx[303] at output port 2 from input port 1
Rx[318] at output port 1 from input port 2
Rx[304] at output port 2 from input port 1
Rx[319] at output port 1 from input port 2
Rx[305] at output port 2 from input port 1
Rx[306] at output port 2 from input port 1
Rx[320] at output port 1 from input port 1
Rx[307] at output port 2 from input port 2
Rx[308] at output port 2 from input port 1
Rx[309] at output port 2 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_input_queued# |
```

Figure 14: Testing of BothtoBoth path

8 Testing of Output queued switch

```
root@cf02f40d9b70:/home/EE789/switch_2x2_output_queued# ./bin/testbench_vhdl s
tdout 1to1 2>&1 | tee log1.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 1 from input port 1
Rx[2] at output port 1 from input port 1
Rx[3] at output port 1 from input port 1
Rx[4] at output port 1 from input port 1
```

```
Rx[428] at output port 1 from input port 1
Rx[429] at output port 1 from input port 1
Rx[430] at output port 1 from input port 1
Rx[431] at output port 1 from input port 1
Rx[432] at output port 1 from input port 1
Rx[433] at output port 1 from input port 1
Rx[434] at output port 1 from input port 1
Rx[435] at output port 1 from input port 1
Rx[436] at output port 1 from input port 1
Rx[437] at output port 1 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_output_queued# |
```

Figure 15: Testing of 1to1 path

```
root@cf02f40d9b70:/home/EE789/switch_2x2_output_queued# ./bin/testbench_vhdl s
tdout 1toBoth 2>&1 | tee log3.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 2 from input port 1
Rx[1] at output port 1 from input port 1
Rx[2] at output port 2 from input port 1
Rx[3] at output port 2 from input port 1
Rx[4] at output port 2 from input port 1
Rx[5] at output port 2 from input port 1
Rx[2] at output port 1 from input port 1
Rx[3] at output port 1 from input port 1
```

```
Rx[285] at output port 2 from input port 1
Rx[286] at output port 2 from input port 1
Rx[287] at output port 2 from input port 1
Rx[313] at output port 1 from input port 1
Rx[314] at output port 1 from input port 1
Rx[288] at output port 2 from input port 1
Rx[289] at output port 2 from input port 1
Rx[290] at output port 2 from input port 1
Rx[291] at output port 2 from input port 1
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_output_queued# |
```

Figure 16: Testing of 1toBoth path

```
root@cf02f40d9b70:/home/EE789/switch_2x2_output_queued# ./bin/testbench_vhdl s
tdout BothtoBoth 2>&1 | tee log4.txt
Info:pThreadUtils: started thread output_port_1_receiver
Info:pThreadUtils: started thread output_port_2_receiver
Info:pThreadUtils: started thread input_port_1_sender
Info:pThreadUtils: started thread input_port_2_sender

Rx[1] at output port 1 from input port 2
Rx[1] at output port 2 from input port 1
Rx[2] at output port 2 from input port 1
Rx[3] at output port 2 from input port 1
Rx[2] at output port 1 from input port 1
Rx[4] at output port 2 from input port 2
Rx[3] at output port 1 from input port 2
Rx[5] at output port 2 from input port 1
Rx[4] at output port 1 from input port 1
Rx[6] at output port 2 from input port 2
```

```
Rx[407] at output port 1 from input port 1
Rx[397] at output port 2 from input port 2
Rx[408] at output port 1 from input port 1
Rx[398] at output port 2 from input port 2
Rx[409] at output port 1 from input port 2
Rx[399] at output port 2 from input port 1
Rx[410] at output port 1 from input port 1
Rx[400] at output port 2 from input port 2
Rx[411] at output port 1 from input port 2
Rx[401] at output port 2 from input port 1
Rx[412] at output port 1 from input port 1
Rx[402] at output port 2 from input port 2
^C
root@cf02f40d9b70:/home/EE789/switch_2x2_output_queued# |
```

Figure 17: Testing of BothtoBoth path

9 Results and Discussion

9.1 Shared Queue Switch

The shared queue switch demonstrates efficient performance in terms of data insertion and retrieval. As soon as a request is received, packet data begins loading into the queue without any noticeable delay (single-cycle response). Similarly, data is delivered to the output ports with minimal latency.

The design was tested under multiple traffic scenarios including:

- 1-to-1
- 1-to-2
- 1-to-both
- 2-to-1
- 2-to-2
- 2-to-both
- Both-to-both

All cases function correctly. However, in the both-to-both scenario, it is observed that Input Port 1 is served more frequently. This behavior arises due to the First-Come-First-Serve (FCFS) policy implemented in the design, which results in implicit priority toward Input Port 1.

9.2 Input-Queued Switch

The input-queued switch (with per-input buffering) was successfully tested across all routing scenarios, and correct functionality was verified.

The design exhibits:

- No delay in packet loading at the input side
- Correct routing of packets to output ports

However, a delay of approximately 3–4 clock cycles is observed at the output stage when an output request signal is applied. This delay occurs due to the longer control path:

Output Request → Output Mux → Output Demux → Queue

This additional latency is a consequence of the multi-stage arbitration and routing mechanism. Despite this delay, the system operates correctly with no functional errors.

9.3 Output-Queued Switch

The output-queued switch demonstrates optimal performance among all three architectures.

- All routing paths function correctly
- No delay is observed during packet loading
- No delay is observed during packet transmission to output ports

Since packets are directly stored in output-specific queues, routing complexity is reduced at the output stage, leading to faster data delivery.

9.4 Overall Comparison

Based on the implemented designs, the following observations can be made:

- The shared queue switch allows communication with only one input at a time due to its centralized buffering.
- Both input-queued and output-queued switches support simultaneous communication with multiple ports, improving throughput.
- The input-queued switch requires less hardware compared to the output-queued switch, as it uses fewer queues.
- The output-queued switch provides the best performance in terms of latency but requires higher hardware resources.